

HC-91 Emulator

User Manual

An emulator for the I.C.E. Felix HC family
(HC-85 / HC-90 / HC-91 / HC-128 / HC-2000)

June 2026

CPU	Z80A (MMN80CPU) @ 3.5 MHz, cycle-exact
Video	256×192, 15 colours, beam-accurate, floating bus
Sound	1-bit beeper; AY-3-8912 on the HC-128
Storage	cassette (.tap/.tZX, pulse-exact), floppy + CP/M on the HC-2000
Build	plain C99, zero dependencies (<code>make</code>)

Contents

1	Introduction	2
2	Quick start	2
3	The emulated machines	2
4	Interactive use: the SDL frontend	3
4.1	Keyboard	3
4.2	Emulator controls	3
4.3	Game controllers	4
5	Loading software from tape	4
5.1	The two loading paths	4
5.2	Sampled recordings (.wav)	4
5.3	Multi-load games and the cassette deck	4
5.4	Saving to tape	5
5.5	TZX block support	5
6	The HC-128: banked RAM and AY sound	5
7	The HC-2000: floppy disks and CP/M	5
7.1	Disk images	5
7.2	HC BASIC disks	6
7.3	Booting CP/M 2.2	6
8	Snapshots, screens and input recordings	6
8.1	Snapshots	6
8.2	RZX input recordings	6

9 Scripting and automation	7
9.1 Typing: <code>--type</code> and <code>--keys</code>	7
9.2 Output channels	7
10 The debugger	7
10.1 Command-line flags	8
10.2 Monitor commands	8
11 The game library	8
12 Accuracy, testing and the suite	8
13 Complete option reference	9
14 Troubleshooting	10

1 Introduction

`hc91emu` emulates the Romanian I.C.E. Felix *HC* home computers — ZX Spectrum-compatible machines produced in Bucharest from 1985 onward. The primary model is the **HC-91**, whose genuine ROM differs from the Sinclair 48K ROM in only 50 bytes, making it fully compatible with the Spectrum software catalogue. The **HC-85** and **HC-90** are banner variants; the **HC-128** adds 128K of banked RAM and an AY-3-8912 sound chip; the **HC-2000** adds a floppy-disk interface and boots CP/M 2.2.

The emulator has two personalities:

- A **headless, scriptable** mode (the default): you state how many 50 Hz frames to run, schedule keypresses at given frames, and collect results as PNG screenshots, OCR'd screen text, raw framebuffers, WAV audio recordings, snapshots or input recordings. Everything is deterministic, which makes it ideal for automated testing — the emulator's own test suite is built on it.
- An **interactive SDL2** mode (`--sdl`): a real window at 50 Hz with live audio, host keyboard mapping, game-controller support, turbo, pause, and cassette controls. The SDL library is loaded at *run time* with `dlopen`, so building the emulator requires nothing beyond a C compiler.

Accuracy highlights: per-M-cycle memory/IO contention, beam-accurate rendering (mid-frame border effects and per-scanline multicolour are correct), the undocumented Z80 behaviours (MEMPTR, the Q register, interrupted block instructions), exact tape pulse timing, and a drift-free 69 888-T-state frame. The core passes `zexdoc`, `zexall`, Patrik Rak's `z80full/z80ccf/z80memptr` (160/160 each) and all 162,000 `SingleStepTests` bus-level vectors.

2 Quick start

```
tools/get_roms.sh  # fetch the ROM dumps (sha256-verified) if absent
make               # build (no dependencies)
make test          # run the test suite

# boot to BASIC and look at the screen
./build/hc91emu --frames 250 --text

# play a game, interactively, with sound
./build/hc91emu --sdl game.tap --autoload
```

A single optional positional argument names the file to load, dispatched by extension: `.tap`, `.tzx`, `.wav` (tape), `.sna`, `.z80`, `.szx` (snapshots), `.scr` (raw screen), `.rzx` (input recording).

On Debian and Ubuntu the packaged release (built by `tools/release_deb.sh`) installs a system-wide `hc91emu` with the ROMs in `/usr/share/hc91emu`; the emulator uses that location automatically whenever a default ROM is not found relative to the current directory:

```
sudo apt install ./hc91emu_1.0.0_amd64.deb
hc91emu --frames 250 --text          # boots; no source tree needed
```

3 The emulated machines

Select with `--machine`; each machine has a sensible default ROM from `roms/` (override with `--rom`).

Machine	Description
hc91	Default. 48K Spectrum-compatible. Port 0x7E bit 0 pages a separate low 16K RAM bank over the ROM — the CP/M mode of the real machine, entered with RANDOMIZE USR 14446.
hc85, hc90	The earlier 48K models (banner-variant genuine ROMs).
48k	A standard Sinclair ZX Spectrum 48K.
hc128	HC-91-derived 128K machine: RAM banks 0–7 at 0xC000 via port 0x7FFD (shadow screen in bank 7, ROM select honoured with <code>--rom1</code> , lock bit; odd banks contended) and an AY-3-8912 at 0xFFFFD/0xBFFD mixed into the audio output.
hc2000	48K-class machine plus the HC disk interface: i8272 floppy controller, IF1-style shadow ROM with 16K interface RAM, and a system latch that can map the CP/M ROM and boot CP/M 2.2 from disk (section 7).

4 Interactive use: the SDL frontend

```
./build/hc91emu --sdl game.tap --autoload
```

opens a resizable 640×480 window running at 50 Hz with live beeper (and AY) audio; pacing is audio-clocked. The frontend needs only the SDL2 *runtime* library (`libSDL2-2.0.so.0`).

4.1 Keyboard

Letters, digits, **Enter** and **Space** map directly to the Spectrum matrix.

Host key	Spectrum meaning
Shift	CAPS SHIFT
Ctrl or left Alt	SYMBOL SHIFT
Backspace	DELETE (CAPS+0)
Esc	BREAK (CAPS+SPACE)
arrows	BASIC cursors (CAPS+5/6/7/8); plain 5/6/7/8 with <code>--joy-type cursor</code> ; Kempston when the interface is attached
, . ; ' - = /	the corresponding SYMBOL SHIFT symbols

4.2 Emulator controls

Key	Action
Tab	turbo while held (no pacing)
F2	quick state save (one <code>.szx</code> slot, see below)
F4	quick state load
F5	pause / resume the emulation
F6	tape: manual play/stop
F7	tape: rewind (paused at the start)
F8	tape: swap cassette side (with <code>--tape-b</code> ; toggles)
F11	fullscreen on/off (aspect kept, letterboxed)
F10 / close	quit

The quick save slot is a single `.szx` file at `$XDG_STATE_HOME/hc91emu/quick.szx` (by default `~/.local/state/hc91emu/quick.szx`; `%APPDATA%\hc91emu` on Windows), so it works no matter which directory the emulator was started from. The initial window is 640 × 480; `--scale N`

picks another multiplier (1–8), and the window can be resized or toggled fullscreen freely — the image keeps its 4:3 aspect.

Drag and drop: dropping a tape, snapshot, screen or RZX file onto the window loads it. Tapes get a clean machine reset followed by the usual autoloading typing (like `--autoload`); snapshots and screens apply on the spot.

4.3 Game controllers

The first SDL game controller drives the Kempston interface (d-pad, left stick, and the A button as fire); plugging one in attaches the interface automatically. On the keyboard, with `--kempston`, arrows + **right Alt** drive Kempston instead of the cursor keys.

5 Loading software from tape

5.1 The two loading paths

Instant (default). LOAD requests are satisfied by a ROM trap at LD-BYTES: blocks appear immediately. Works for normal `.tap` files and the standard/turbo data blocks of `.tzz` files.

Pulse-level (`--real-tape`). The whole tape is compiled to an EAR pulse stream with exact T-state timing and ‘played’. Required for custom and turbo loaders, loading stripes, and anything timing-sensitive. `--play-at N` chooses when PLAY is pressed (default: frame 320 with `--autoload`).

`--autoload` types LOAD "" + ENTER at frame 250 so tapes start by themselves.

5.2 Sampled recordings (`.wav`)

A `.wav` file — a digitized recording of a genuine cassette, e.g. captured from a real HC-91 or a preserved audio archive — loads through the pulse player directly. The samples get a DC-removal pass (an exponential moving average) and a Schmitt trigger at one eighth of the peak deviation; the resulting edge-to-edge times become tape pulses, so a recording loads exactly like the cassette it came from. `--real-tape` is implied. Plain PCM is accepted (8- or 16-bit, mono or stereo, 4–192 kHz); silences longer than a second are treated as block boundaries, so multi-load auto-pause works on sampled tapes too. The bundled `tests/tap2wav` renders a `.tap` into such a recording if you want to experiment.

5.3 Multi-load games and the cassette deck

Games that load levels on demand (“stop the tape”, “turn the tape over”) work in `--real-tape` mode:

- The player **auto-pauses** at a block boundary whenever no loader is actually polling the tape port. (Reads made by the ROM keyboard scanner are recognised and ignored, so a “press any key” prompt does not keep the tape rolling past data.)
- It **auto-resumes** as soon as loading restarts.
- TZX **stop-the-tape** markers pause unconditionally.
- At the **end of the tape**, if a loader is still searching, the tape rewinds itself (“rewind tape” prompts).

- **Two-sided originals:** attach the other side with `--tape-b FILE` and press **F8** when the game asks you to turn the tape over (headless: `--swap-at N`). Each swap toggles the sides, rewind and paused.

```
./build/hc91emu --sdl shooter/p47_side_a.tzx --autoload --real-tape \
    --tape-b shooter/p47_side_b.tzx
# F8 at "PLEASE TURN TAPE OVER", any key to continue;
# hold Tab to fast-forward the long level searches
```

The tape state is reported on stderr (tape: paused at block 5/23 ..., tape: resumed, tape: end of tape - rewind).

5.4 Saving to tape

`--save-tape FILE` captures BASIC SAVE output (via the SA-BYTES trap) as a byte-exact .tap.

5.5 TZX block support

Standard/turbo data (0x10/0x11), pure tone / pulse sequences / pure data (0x12–0x14), **CSW recordings** (0x18; RLE and zlib Z-RLE), **generalized data** (0x19; symbol alphabets, PRLE pilot runs, polarity flags), pause/stop (0x20), groups (0x21/0x22), loops (0x24/0x25), stop-if-48K (0x2A), and the information blocks (0x30–0x35, 0x5A).

6 The HC-128: banked RAM and AY sound

The HC-128 keeps the 48K BASIC (its ROM is HC-91-derived) and adds the Spectrum-128 hardware interfaces, so banking and sound are driven with OUTs:

```
# select RAM bank N at 0xC000 (keep bit 4 set): OUT 32765, 16+N
# move the stack down first or the switch pulls it from under BASIC!
CLEAR 32767: OUT 32765,17: POKE 49152,42: OUT 32765,16
```

The AY-3-8912 sits at the standard ports: OUT 65533,reg selects, OUT 49149,val writes (reading IN 65533 returns the selected register). All three tone channels, noise and envelopes are emulated with the measured volume curve and mixed sample-accurately with the beeper into --wav and the SDL audio.

128K .z80 (hardware mode 3) and .szx (machine id 2) snapshots save and load the full bank set, the 0x7FFD latch and the AY registers. 48K snapshots load into a locked, USR0-style bank layout, so ordinary games run on the HC-128 too.

7 The HC-2000: floppy disks and CP/M

7.1 Disk images

`--disk FILE` inserts drive A, `--disk-b` drive B. Raw images are recognised by size:

Size	Geometry	Filesystem
655 360	80×2×16×256	HC BASIC 3.5" (640K)
737 280	80×2×9×512	CP/M 2.2 3.5" (720K, 4 boot tracks)
327 680	40×2×16×256	HC BASIC 5.25"
368 640	40×2×9×512	CP/M 2.2 5.25"

Writes are buffered in memory and written back to the image file when the emulator exits.

7.2 HC BASIC disks

The interface presents Sinclair Interface 1 syntax to BASIC:

CAT 1	list the disk in drive 1
LOAD *"d";1;"run"	load a program from disk

(CAT is an extended-mode keyword: press CAPS+SYM, then SYM+9.)

7.3 Booting CP/M 2.2

```
./build/hc91emu --machine hc2000 --boot-cpm --disk cpm22-hc.img \
--sdl
```

boots from the system tracks to 56k CP/M ver 2.2/2D and the A> prompt; DIR, applications (WordStar, Turbo Pascal, MBASIC...) and the rest of CP/M work. The authentic route from BASIC is RANDOMIZE USR 14446, which selects the CP/M ROM and reboots into it. The CP/M console renders with its own font at a relocated screen — use screenshots rather than `--text` there.

Set `HC91_FDC_DEBUG=1` to log every i8272 command/result on stderr.

8 Snapshots, screens and input recordings

8.1 Snapshots

Load by passing the file; save with `--save-sna`, `--save-z80`, `--save-szx` after the run.

.sna	48K, load + save. Save needs SP outside ROM (the PC is pushed); on hc128 only the current 48K view can be represented.
.z80	v1/v2/v3 load, incl. RLE compression and 128K files; saves as v2 with uncompressed pages (48K or full 128K with latch+AY).
.szx	zx-state v1.4 load + save (48K and 128K). Compressed RAM pages in files from other emulators are inflated by a built-in DEFLATE decoder — still zero external dependencies.
.scr	raw 6912-byte screens; loading parks the CPU so the picture stays.

A HALT state survives snapshots (PC is re-pointed at the HALT, or the SZX HALTED flag is used).

8.2 RZX input recordings

```
# record a session (embeds a .z80 of the starting state)
./build/hc91emu --frames 450 --type 'p2+3*4\n@260' --rzx-record calc.rzx
# replay: inputs come from the file, no key events needed
./build/hc91emu calc.rzx --frames 460 --text      # prints 14 again
```

Replay is bit-exact: each frame runs for the recorded number of R/M1 fetches and every port read returns the recorded byte. When the recording ends, the machine continues running live. Compressed RZX files from other emulators replay; “protected” (encrypted) ones are rejected.

9 Scripting and automation

Typical recipe: schedule input, run N frames, collect output, assert.

```
./build/hc91emu --frames 450 --type 'p2+3*4\n@260' --text \
--screenshot out.png --wav out.wav --save-z80 out.z80
```

9.1 Typing: --type and --keys

--type "STRING@FRAME" types a string with BASIC key mapping: lowercase letters press the bare key (producing K-mode keywords at the start of a statement), digits and common symbols work as expected, \n is ENTER. Keys are held for 6 frames with 6-frame gaps (≈ 8 characters/second), so leave room between scheduled items. --keys "FRAME:NAMES" presses one raw chord, e.g. "272:CAPS+SYM"; names are A–Z, 0–9, ENTER, SPACE, CAPS, SYM. Both repeat up to 32 times.

Keyword cheat sheet (48K BASIC keyword entry):

K-mode (statement start)	p=PRINT	j=LOAD	o=POKE	s=SAVE
	t=RANDOMIZE	x=CLEAR	f=FOR	i=INPUT
EXTEND mode	press CAPS+SYM first (one --keys entry), then: plain			
	o=PEEK, plain i=CODE, plain l=USR; with SYM:			
	SYM+0=OUT, SYM+I=IN, SYM+Z=BEEP, SYM+9=CAT			

Example — OUT from BASIC (used by the HC-128 tests):

```
--keys '260:CAPS+SYM' --keys '272:SYM+0' --type '65533,0\n@284'
```

9.2 Output channels

--text	24×32 screen OCR against the ROM font (inverse video recognised; unmatched cells print ?)
--screenshot	320×240 PNG of the final frame (beam-painted)
--fb-dump	the same frame as raw RGBA bytes
--wav	44.1 kHz mono WAV of the whole run
--trace-frames	frame/PC progress on stderr

Exit status is 0 on success, 1 on argument/file errors — suitable for shell test scripts (see `tests/run_tests.sh` for several hundred lines of worked examples).

10 The debugger

A scriptable monitor with a full Z80 disassembler (documented and undocumented opcodes). Addresses/ports are hex (\$ or 0x prefixes accepted); counts are decimal.

10.1 Command-line flags

<code>--monitor</code>	stop in the monitor before the first instruction
<code>--break ADDR</code>	PC breakpoint (repeatable)
<code>--watch ADDR</code>	memory <i>write</i> watchpoint
<code>--rwatch ADDR</code>	memory <i>read</i> watchpoint (fires on opcode fetches too)
<code>--pwatch PORT</code>	I/O watchpoint; values $\leq$ FF match the low byte, larger values match the full 16-bit port
<code>--debug "c;c;q"</code>	scripted monitor commands, consumed one per stop; falls back to stdin, then auto-continues
<code>--trace FILE</code>	per-instruction trace (pre-execution state)

10.2 Monitor commands

<code>r, regs</code>	registers, decoded flags, frame-relative T-state, frame number, total T
<code>d, dis [ADDR [N]]</code>	disassemble (default PC, 8 instructions; <code>pc/sp/hl</code> accepted as ADDR)
<code>m, mem ADDR [N]</code>	hex + ASCII dump (default 64 bytes)
<code>s, step [N]</code>	execute N instructions, stop again
<code>c, cont</code>	continue
<code>b / w / wr / wp [ADDR]</code>	toggle breakpoint / write / read / port watch; bare = list
<code>t, trace FILE off</code>	tracing on/off
<code>q, quit</code>	exit immediately (skips end-of-run saves)
<code>h, help</code>	summary

An empty input line repeats the last command. Example session:

```
$ ./build/hc91emu --frames 3 --break 11CB \
    --debug "regs;dis pc 3;mem 0000 16;step 3;regs;cont"
*** breakpoint at $11CB
11CB: 47          LD B,A
dbg> regs
PC=11CB AF=0044 BC=0000 DE=FFFF HL=0000 ... T=28/69888 frame=0
...
```

11 The game library

`tools/get_library.sh` downloads 36 period classics from the World of Spectrum archive into `software/library/`, classified by genre (platform, arcade, isometric, shooter, puzzle, adventure, sports), including 128K AY versions of Cybernoid and Tetris for the HC-128 and the authentic two-sided P-47 cassettes. `tools/verify_library.sh` smoke-loads every title and stores screenshots under `tests/out/library/`. The library README carries the full index (title, year, publisher, file). The files are copyrighted period software: the scripts make the library reproducible, nothing under `software/` is committed.

12 Accuracy, testing and the suite

`make test` runs the suite: timing and contention unit tests, the disassembler tests, zexdoc, boot/BASIC checks, the beam-renderer and floating-bus acceptance tests, snapshot round-trips, the tape and TZX matrix (including multi-load pause/resume), joysticks, CP/M paging, the debugger, SingleStepTests vectors (162,000 bus-level cases, fetched once by `tests/get_vectors.sh`),

the SDL frontend under dummy drivers, the HC-128 and HC-2000 machines (CP/M boots against a golden screen) and a library smoke test. `make test-full` adds zexall; `RUN_Z80TEST=1` re-runs Rak's in-machine suites.

Environment variables: `HC91_FDC_DEBUG=1` (i8272 command log), `HC91_TAPE_DEBUG=1` (tape block progression), `SDL_VIDEODRIVER=dummy` `SDL_AUDIODRIVER=dummy` (headless SDL).

13 Complete option reference

General

<code>--machine M</code>	hc91 (default), hc85, hc90, 48k, hc128, hc2000
<code>--rom FILE</code>	main ROM override (default per machine)
<code>--rom1 FILE</code>	hc128 second ROM (default: copy of ROM 0)
<code>--rom-if1 FILE</code>	hc2000 interface ROM (default roms/hc2ki1.rom)
<code>--frames N</code>	frames to run headless (default 300; 50 = 1 s)
<code>--turbo</code>	accepted, no-op (headless never paces)
<code>--no-floating-bus</code>	unattached ports read 0xFF

Output

<code>--screenshot FILE</code>	PNG of the final frame (320×240)
<code>--text</code>	print the final screen as ROM-font OCR text
<code>--fb-dump FILE</code>	final frame as raw RGBA
<code>--wav FILE</code>	44.1 kHz mono WAV of the run
<code>--save-sna FILE</code>	save .sna after the run
<code>--save-z80 FILE</code>	save .z80 v2 after the run
<code>--save-szx FILE</code>	save .szx after the run
<code>--save-scr FILE</code>	save the raw 6912-byte screen

Input scripting

<code>--type "S@F"</code>	type string S from frame F (BASIC mapping, \n = ENTER; repeatable)
<code>--keys "F:NAMES"</code>	raw chord at frame F, e.g. 100:CAPS+1 (repeatable)
<code>--autoload</code>	type <code>LOAD ""</code> at frame 250
<code>--joy "F-G:DIRS"</code>	hold U/D/L/R/F over frames F..G (repeatable)
<code>--joy-type T</code>	kempston (default) / sinclair / cursor
<code>--kempston</code>	attach the Kempston interface (port 0x1F)

Tape

<code>--real-tape</code>	pulse-level playback (multi-load auto pause/resume/rewind)
<code>--play-at N</code>	press PLAY at frame N (default 320 with autoload)
<code>--tape-b FILE</code>	the other cassette side (F8 / <code>--swap-at</code> toggles)
<code>--swap-at N</code>	insert <code>--tape-b</code> at frame N (headless)
<code>--save-tape FILE</code>	capture SAVE output as byte-exact .tap

Disks and CP/M (hc2000)

<code>--disk FILE</code>	drive A image (raw .img/.dsk; written back on exit)
<code>--disk-b FILE</code>	drive B image
<code>--boot-cpm</code>	reset into the CP/M boot ROM

Recordings

<code>--rzx-record FILE</code>	record the session as a replayable .rzx
--------------------------------	---

SDL frontend

<code>--sdl</code>	interactive window, 50 Hz, live audio
<code>--sdl-frames N</code>	auto-quit after N frames
<code>--scale N</code>	initial window size multiplier 1–8 (default 2)
<code>--trace-frames</code>	frame/PC progress on stderr

Debugger

<code>--monitor</code>	stop before the first instruction
<code>--break ADDR</code>	PC breakpoint (hex; repeatable)
<code>--watch ADDR</code>	memory write watchpoint
<code>--rwatch ADDR</code>	memory read watchpoint (incl. fetches)
<code>--pwatch PORT</code>	I/O watchpoint ($\leq \text{FF}$ = low byte)
<code>--debug "C;C;..."</code>	scripted monitor commands
<code>--trace FILE</code>	per-instruction trace to FILE

14 Troubleshooting

A game freezes at its loading screen. Try `--real-tape`: it probably uses a custom/turbo loader the instant trap cannot serve.

A multi-load game asked me to flip the tape and stopped. Use the two-sided release with `--tape-b` and press **F8** at the prompt; for single-file tapes the player pauses and resumes automatically — watch the `tape:` messages on stderr, and use **F6/F7** for manual control. Hold **Tab** through long level searches.

--text prints only ? characters. The program draws with a custom font (e.g. the CP/M console); use `--screenshot` instead.

A game ignores the joystick. Check `--joy-type`; for Kempston games also pass `--kempston` (the port is detached by default so that floating-bus games work).

CP/M shows a black screen in the first seconds. Normal: the boot ROM relocates the video memory and clears it before the banner appears (~2500 frames to the `A>` prompt).

Arkanoid freezes at round start. You passed `--no-floating-bus`; the game needs the floating bus (default).

See also: `man docs/hc91emu.1`, `README.md`, `ROADMAP.md` (development history),
`software/library/README.md` (game index).